Biçimsel Diller ve Otomatlar

Abdullah Erhan Akkoyun

Güz Dönemi Notları

1. Sonlu özdevinirler

Sonlu Özdevinirler (FA [Finite automata]) Modeli

* Sonlu özdevinirler (FA) modeli, kesikli giriş ve çıkışları olan matematiksel modeldir. Bu model birçok yazılım ve donanım sistem modellemesinde kullanılır.

Örn: Metin düzenleyiciler (text editor), derleyicilerin belirli kesimlerini

* Sonlu özdevinirler modeli birçok türü vardır. Sonlu özdevinirler öncelikle;

1. Sonlu durum tanıyıcı (finite state recognizer (FSR)) → DFA / NFA
2. Çıkış üreten → Mealy / Moore

modelleri olarak sınıflandırılır.

Deterministik Sonlu özdevinir modeli (DFA)

* Temel modelde, deterministik sonlu özdevinir bir beşli olarak tanımlanır.

$DFA = \langle Q, \Sigma, \delta, q_0, F \rangle$

Q = Sonlu Durum içeren Durumlar Kümesi

Σ = Giriş Simgesi

δ = Durum geçiş işlevi $Q \times \Sigma$

q_0 = Başlangıç Durumu

F: Son Durum (Uç Durum) $\rightarrow F \subseteq Q$

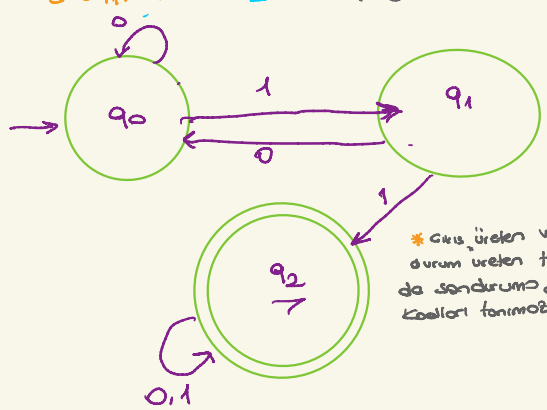
- $M = \langle Q, \Sigma, \delta, q_0, F \rangle$
  - $Q = \{q_0, q_1, q_2\}$
  - $\Sigma = \{0, 1\}$
  - $F = \{q_2\}$

$\delta = \delta(q_0, 0) = q_0$
$\delta(q_0, 1) = q_1$
$\delta(q_1, 0) = q_0$
$\delta(q_1, 1) = q_2$

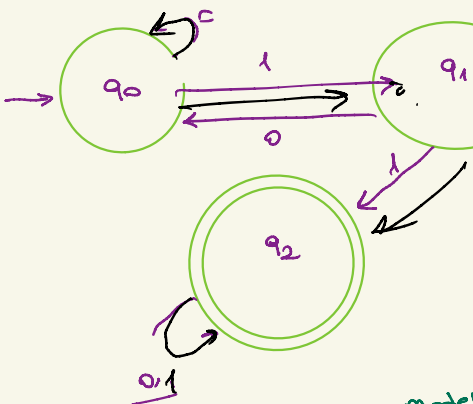
$\delta(q_2, 1) = q_2$
$\delta(q_2, 0) = q_2$

* Değerleri uygulamaya Başlangıç Stateten başlanır.

	0	1
Başlangıç → q_0	q_0	q_1
q_1	q_0	q_2
→ Final State q_2	q_2	q_2



* Çıkış üreten ve sonlu durum üreten tüm makinaları da sonduruma götürmeyen koşulları tanımlar.



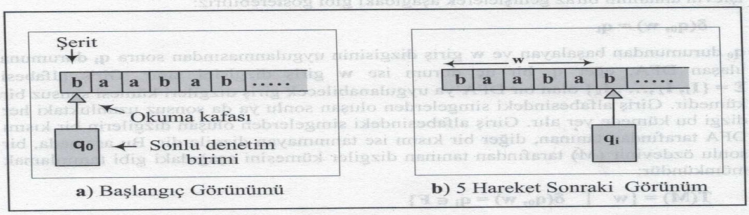
→ 0110 uygulayalım → makine bu girişi algılar.

→ algılama durumlarında makinanın bitmesi gerekmektedir.

→ 1010 → makine durmadığı için bu girişi algılamaz.

DFA modelinde Şeritli Makine Modeli

* Tanımlandığı biçimiyle deterministik sonlu özdevinir modeli matematiksel bir modeldir. Deterministik sonlu özdevinir oluşturmak, durumlardan birini başlangıç durumu olarak seçmek, durumların bir kesimini UG (tanıyan) durumları nitelemek ve geçiş işlevi tanımlamak yeterlidir.



Çizim 1.2. DFA'nın Şeritli Makine Modeli

Çizim 1.2.a'da görüldüğü gibi DFA'nın makine modeli 3 elemandan oluşmaktadır.

- Hücrelerden oluşan ve her hücresinde bir giriş simgesi bulunan bir miknatıslı şerit. Yalnız okunabilen şeridin sağ ucu sonsuzdur. Başlangıçta şerit üzerinde giriş simgelerinden oluşan bir dizgi kayıtlıdır.
- Bir sonlu denetim birim (SDB). SDB'nin sonlu sayıda durumu vardır. Bu durumlardan biri başlangıç durumudur ve SDB başlangıçta bu durumda bulunur.
- Bir okuma kafası. Şeridin okunması soldan sağa doğru, tek yönlü gerçekleşir. Belirli bir anda okuma kafası şeridin hücrelerinden biri üzerinde bulunur ve üzerinde bulunduğu hücrede kayıtlı simgeyi okuyabilir.

Çizim 1.2'deki modele göre, DFA (makine) aşağıda açıklanan biçimde çalışır:

- Başlangıçta şerit üzerinde bir giriş dizgisi kayıtlıdır ve okuma kafası şeridin ilk (en soldaki) hücresi üzerindedir.
- Makinenin her hareketinde aşağıdaki işlemler yapılır:
  1. Şeritten bir simge okunur.
  2. Okuma kafası bir sağdaki hücreye geçer.
  3. Sonlu denetim birimi bir sonraki duruma geçer.
- Belirli sayıda hareket sonunda okuma kafası belirli bir hücrenin üzerinde, sonlu denetim birimi ise belirli bir durumda (q_i) bulunur. Okuma kafasının üzerinde bulunduğu hücrenin solundaki hücrelerde yer alan dizgiyi **w** olarak adlandıralım. Eğer q_i bir uç durumsa, makine **w**'yi tanır (Çizim 1.2.b).

Deterministik olmayan sonlu özdevinir (NFA) modeli

* Sözlü olarak düzgün deyimle tanımlanan belirli dizgiler kümesini tanıyan DFA'nın deterministik geçiş çizeneğini oluşturmak oldukça zordur.

* NFA'da bir giriş simgesi uygulandığında sonucu yo birden fazla yada hiç durum olmasıdır. (yo birden fazla çıktı yada hiç çıktı vermiyecek)

- $M = \langle Q, \Sigma, \delta, q_0, F \rangle$

$Q = \{q_0, q_1, q_2, q_3\}$

$\Sigma = \{0,1\}$

$F = q_3$

$q_0 = q_0$

$\delta = \delta(q_0,0) = \{q_0,1\}$

$\delta(q_0,1) = \{q_0,q_2\}$

$\delta(q_1,0) = \{q_3\}$

$\delta(q_1,1) = \emptyset$

$\delta(q_2,0) = \emptyset$

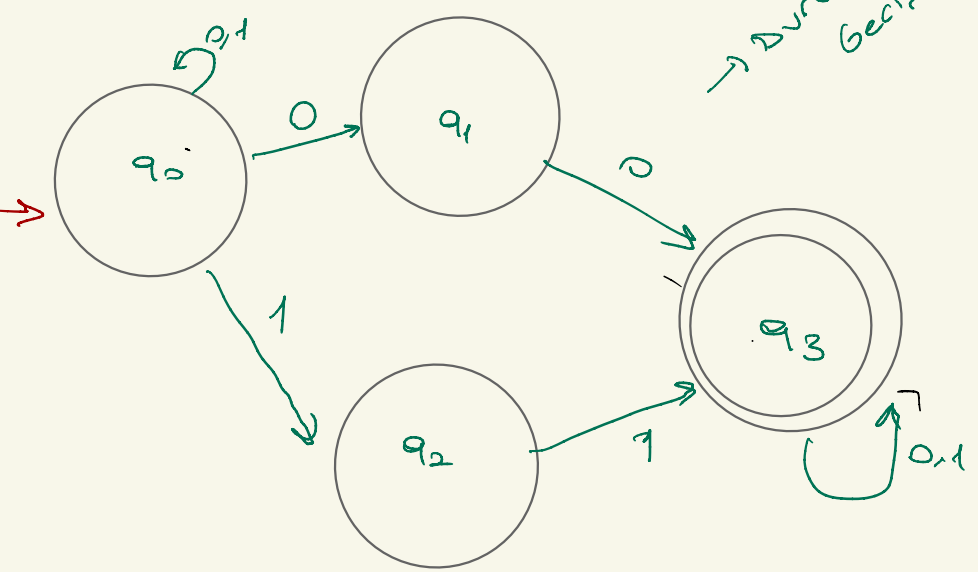
$\delta(q_2,1) = \{q_3\}$

$\delta(q_3,0) = \{q_3\}$

$\delta(q_3,1) = \{q_3\}$

	0	1
q_0	q_0, q_1	q_0, q_2
q_1	q_3	$\emptyset$
q_2	$\emptyset$	q_2
q_3	q_3	q_3

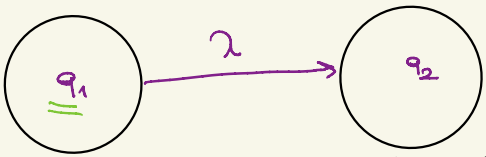
→ Durum Geçiş Tablosudur.



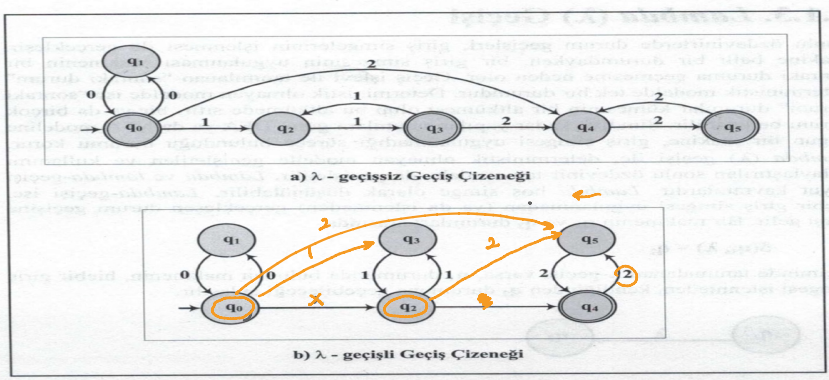
→ Durum Geçiş Diyagramı

Lambda (λ) Geçişi

* Sonlu özdevinirlerde durum geçişleri, giriş simgeleri işlenmesiyle gerçekleşir. Makine belirli bir durumdayken, bir geçiş simgesinin makinenin bir sonraki duruma geçmesine neden olur.
* Bir durumdayken başka bir durumda olma durumudur.



* Lambda geçişi hiçbir simge uygulanmadan gerçekleşen durum geçişidir.
  - → Eğer q_1 başlangıç durum ise q_2 de başlangıç durumudur.
  - → Eğer q_2 son durum ise q_1 de son durumdur.
  - $T(M) = \{0^n 1^m 2^k \mid n \geq 0, m \geq 0, k \geq 0\}$



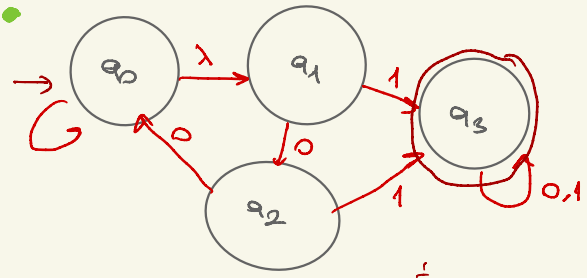
λ Geçişin Kaldırılması

* λ geçişli çizeneği verildiğinde, λ geçişi tek tek yok edercek aş değer bir geçiş çizeneği elde etmek mümkündür. Bu kapsamda q_1 ve q_2 durumları arasında q_1'den q_2'ye λ geçişi varsa;

→

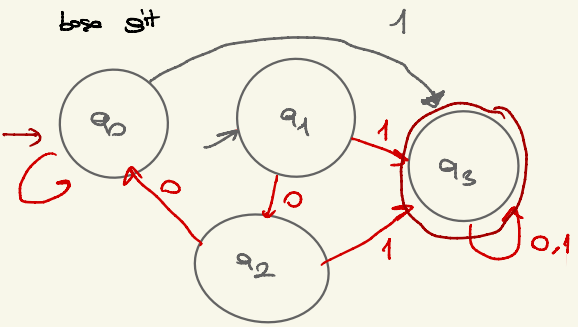
- q_2 durumundan başalayan her durum geçişine ($\forall\ \delta(q_2, a) = q_k$) karşılık q_1 durumundan başlayan ve aynı giriş simgesi ile aynı duruma ulaşan bir durum geçişi ($\delta(q_1, a) = q_k$) eklendikten,
- eğer q_1 durumu başalangıç durumu ise, q_2 durumu da başlangıç durumu yapıldıktan, ve

- eğer q_2 durumu bir uç durum ise, q_1 durumu da uç durum yapıldıktan sonra, λ-geçişi silinebilir.



Londa Geçişli Geçiş Çizneği

*En sondan başla Londa görene kadar başa git



DFA-NFA Denkliği

*Eğer iki model ile tanınabilen kümeler sınıfı aynı ise bu iki model denktir. Yada eğer modellerden biri ile tanınabilen ancak diğeri ile tanımlanamayan kümeler varsa, bu iki model denk değildir.

*Her DFA bir NFA'yı yodenktir. Fakat Her NFA DFA'ya denk olmayabilir.

- $m = <Q, \Sigma, \delta, q_0, F>$

$Q = \{A, B, C\}$

$\Sigma = \{0,1\}$

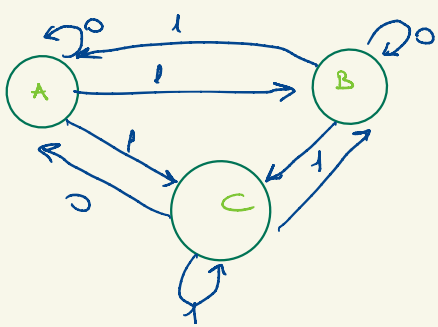
$q_0 = \{A\}$

$F = \{C\}$

$\delta = \delta(A,0) = \{A\}$ $\quad \delta(B,0) = \{B\}$

$\delta(A,1) = \{B,C\}$ $\quad \delta(B,1) = \{A,C\}$

$\delta(C,0) = \{A,B\}$ $\quad \delta(C,1) = \{C\}$



Durum Geçiş Diyagramı

	0	1
→ A	A	BC
B	B	AC
(C)	AB	C

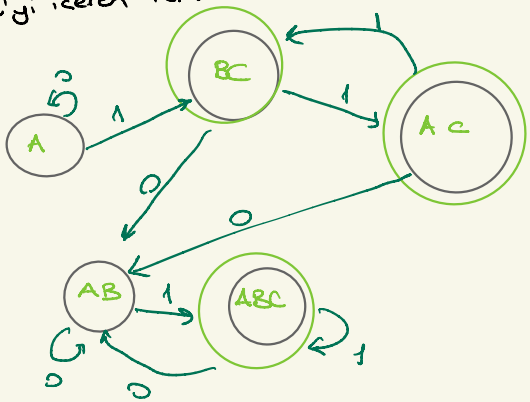
Durum Geçiş Fonksiyonu

DFA →

	0	1
→ A	A	BC
(BC)	AB	AC
AB	AB	ABC
(AC)	AB	BC
(ABC)	AB	ABC

* A'yı içeren tüm ifadeler başlangıç durumu olamaz.

* C'yi içeren tüm durumların hepsi uç durumdur.



→ NFA'nın DFA'ya dönüşmüş halidir.

İki yönlü DFA modeli

→ sağa git

* $\delta(Q \times \Sigma) \rightarrow Q \times \{L, R\}$

↳ sola git

- $M = \langle Q, \Sigma, \delta, q_0, F \rangle$

$Q = \{q_0, q_1, q_2\}$

$\Sigma = \{0, 1\}$

$f = \{q_0, q_1, q_2\}$

$\delta = \delta(q_1, 0) = (q_0, R) \quad \delta(q_1, 1) = (q_2, L)$

$\delta(q_0, 1) = (q_1, R) \quad \delta(q_2, 0) = (q_0, R)$

$\delta(q_1, 0) = (q_1, R) \quad \delta(q_2, 1) = (q_2, L)$

	0	1
→ q_0	q_0, R	q_1, R
q_1	q_1, R	q_2, L
q_2	q_0, R	q_2, L

$ID_1 \rightarrow ID_2$ → Tek yönlü 0 kuma

$ID_1 \xrightarrow{*} ID_2$ → Çok yönlü okuma

a) $w_1 = 1000101$

$(q_0, 1000101) \vdash (1, q_1, 000101) \vdash (10, q_1, 00101) \vdash (100, q_1, 0101) \vdash$
$(1000, q_1, 101) \vdash (100, q_2, 0101) \vdash (1000, q_0, 101) \vdash (10001, q_1, 01) \vdash$
$(100010, q_1, 1) \vdash (10001, q_2, 01) \vdash (100010, q_0, 1) \vdash (1000101, q_1)$

Sonuç : $w_1 = 1000101$ dizgisi $M_{1.7}$ tarafından tanınır.

b) $w_2 = 100110$

$(q_0, 100110) \vdash (1, q_1, 00110) \vdash (10, q_1, 0110) \vdash (100, q_1, 110) \vdash$
$(10, q_2, 0110) \vdash (100, q_0, 110) \vdash (1001, q_1, 10) \vdash (100, q_2, 110) \vdash$
$(10, q_2, 0110) \vdash (100, q_0, 110)$ .....

Sonuç : ID geçişlerinde döngü oluştuğu için $w_2 = 100110$ dizgisi $M_{1.7}$ tarafından tanınmaz.

Giriş dizgilerinin 2DFA tarafından tanınıp tanınmadığını, ID'ler arası geçiş ilişkileri yerine, şerit üzerindeki hareketleri ve durum değişimlerini inceleyerek de bulabiliriz. Bu yöntemle $w_1 = 1000101$ ve $w_2 = 100110$ giriş dizgilerinin $M_{1.7}$ makinesi tarafından tanınıp tanınmadığı aşağıdaki gibi bulunabilir.

a) $w_1 = 1000101$

1	0	0	0	1	0	1

$q_0 \rightarrow q_1 \rightarrow q_1 \rightarrow q_1 \rightarrow q_1 \rightarrow q_2$

$q_0 \rightarrow q_1 \rightarrow q_1 \rightarrow q_2$

$q_0 \rightarrow q_1$

Sonuç : $w_1 = 1000101$ dizgisi $M_{1.7}$ tarafından tanınır.

Moore Makinası

* Moore Makinası, bir altılı olarak tanımlanır.
* Moore makinası, durum düzeyinde çıkış üretir.

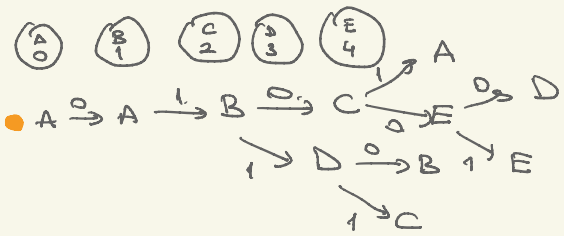
● $M = \langle Q, \Sigma, \Delta, \delta, \omega, \lambda \rangle$ → çıkış fonksiyonu

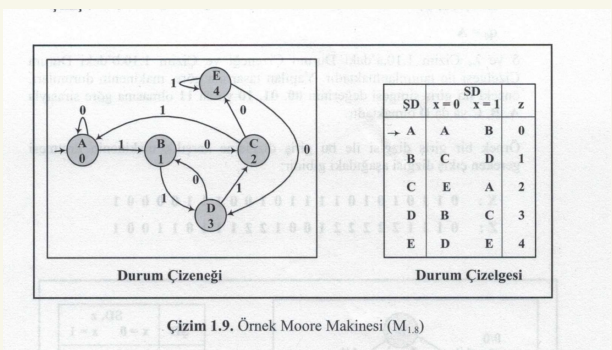
Δ → çıkış alfabesi

● $Z \equiv mod(x,5)$ → x binary geçişi, Z çıkış

$Z = \{0, 1, 2, 3, 4\}$ (A B C D E)

	0	1	Z
A	A	B	0
B	C	D	1
C	D	A	2
D	B	C	3
E	D	E	4





Çizim 1.9. Örnek Moore Makinesi ($M_{1.8}$)

Mealy Makinası

* Geçiş düzeyinede çıkış verir.

$M = \langle Q, \Sigma, \Delta, \delta, \lambda, q_0 \rangle$

! moore ⇒ $Q \to \Delta$
mealy ⇒ $(q \times \Sigma) = \Delta$

● $M = \langle Q, \Sigma, \Delta, \delta, \lambda, q_0 \rangle$

$Q = \{A, B, C, D, E\}$

$\Sigma = \{0, 1\}$

$\Delta = \{0, 1, 2\}$

$q_0 \in A$

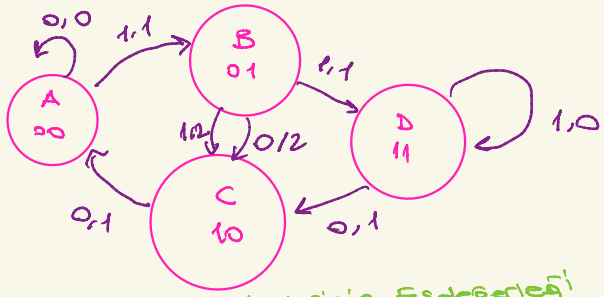
x: 0 1 1 0 1 0 1 0 1 1 1 0 1 0 0 1 1 1 0 0 0 0 1

Z 0 1 1 1 2 2 2 2 1 0 0 1 2 2 1 1 1 0 1 1 0 0 1

	0	1
∞ → A	A, 0	B, 1
01 → B	C, 2	D, 1
10 → C	A, 1	B, 2
11 → D	C, 1	D, 0

x: 0 1 1 0 1 0 1 0 1 1 1 0 1 0 0 1 1 1 0 0 0 0 1

Z 0 1 1 1 2 2 2 2 1 0 0 1 2 2 1 1 1 0 1 1 0 0 1

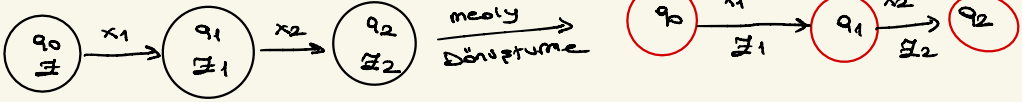


Moore ve Mealy Makinelerinin Eşdeğerliği

* Moore ve mealy makinelerinin ikisinde çıkış üreten özdevinir modelidir. Birer altlı olarak tanımlanan moore ve mealy modellemelerinde altlının salonsal ortaktır.

* Moore makinesi modelinde, her durum için çıkış üretmektedir. Mealy makinesi ise her durum geçişi için bir çıkış simgesi üretmektedir.

I. Moore Eşdeğer Mealy



*

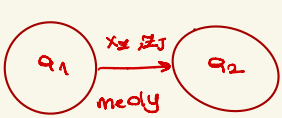
	0	1	Z
A	A	B	0
B	C	D	1
C	E	A	2
D	B	C	3
E	D	E	4

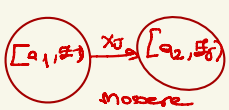
	0	1
A	A,0	B,1
B	C,2	D,3
C	E,4	A,0
D	B,1	C,3
E	D,3	E,4

II. Mealy Eşdeğer Moore

* $M = \langle Q, \Sigma, \delta, q_0, \lambda \rangle \Longrightarrow M_{moore} \langle Q', \Delta, \delta', \lambda', q_0 \rangle$

- $Q' = Q \times \Delta$
- $q_0' = \{q_0, Z\}$ → random çıkış seviyesi
- $\lambda' = [(q_1, Z_k)] = Z_k$
- $\delta' = [(q_1, Z_k), x_j) = \delta[q_i, x_j), \lambda(q_i x_j)$





● $M = \langle Q, \Sigma, \delta, q_0, \lambda \rangle$

$Q = \{A, B, C\}$

$\Sigma = \{0, 1\}$

$q_0 = \{A\}$

$\delta(A,0) = B$

$\delta(A,1) = A$

$\delta(B,0) = B$

$\delta(B,1) = C$

$\delta(C,0) = A$

$\delta(C,1) = C$

$\lambda(A,0) = 0$

$\lambda(A,1) = 1$

$\lambda(B,0) = 1$

$\lambda(B,1) = 1$

$\lambda(C,0) = 0$

$\lambda(C,1) = 0$

q_1 q_2 q_3

* $Q' = Q \times A = \{(A,0), (A,1), (B,0), (B,1), (C,0), (C,1)\}$

$q_0' = [A,0]$ → Önemli olan 0'dan başlamasıdır.

$\lambda'(A,0) = 0$

$\lambda'(A,1) = 1$

$\lambda'(B,0) = 0$

$\lambda'(B,1) = 1$

$\lambda'(C,0) = 0$

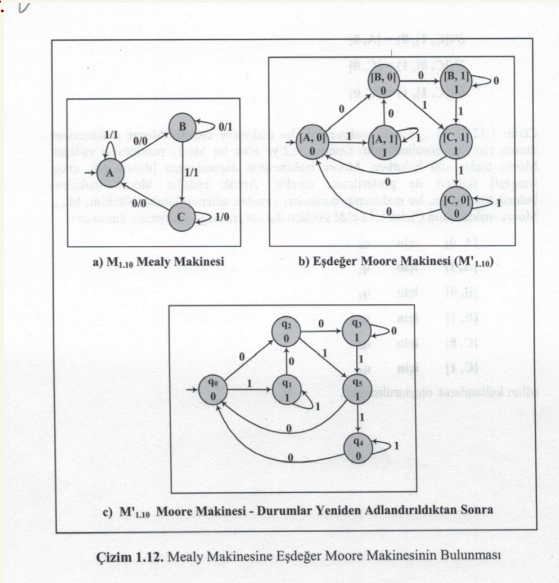
$\lambda'(C,1) = 1$

$\delta'[(A,0),0] = (B,0)$

$\delta((A,0),1) = (A,$

$\delta((A,1),0) = (1$

$\delta((A,1),1) = ($

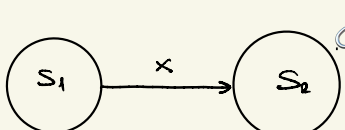


Çizim 1.12. Mealy Makinesine Eşdeğer Moore Makinesinin Bulunması

FA indirgenmesi

	x=0	x=1
→ A	A,0	D,1
B	C,0	E,1
C	G,0	E,1
D	G,0	F,1
E	E,1	C,0
F	B,0	D,1
G	B,0	E,1

Ardıl: ● M makinesin S_1 durumundan x giriş simgesi ile S_2 geciyorsa;



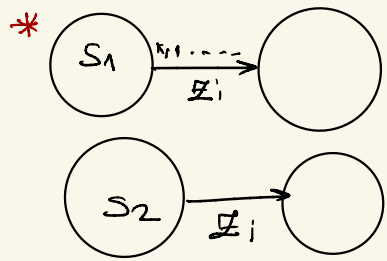
Öncel: ● A $\xrightarrow{1}$ D $\xrightarrow{0}$ G $\xrightarrow{1}$ E

A $\xrightarrow{101}$ E

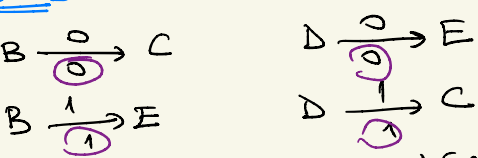
Tek adımda girişle gitmek varken birden fazla duraga uğramadır.

	x=0	x=1
→ A	A	—
B	F, G	—
C	B	E
D	—	A, F
E	E	B, G, G
G	C, D	—
F	—	D

Denk Durum



Örn:1 B ve D 1 denk mi



II. B ve D 2 denk mi

	B	D
00	G, 0	B, 0
01	E, 1	E, 1
10	C, 0	B, 0
11		

Bir giriş simgesi uygulodiğimizde gittiğimiz yerin önemi yoktur. Sadece inputa karşılık gelen output eşit ise bunlar denktir

* Farklı çıkış verdiği için B ve D iki denk değildir.

n denk n+1 ayırt edilebilir

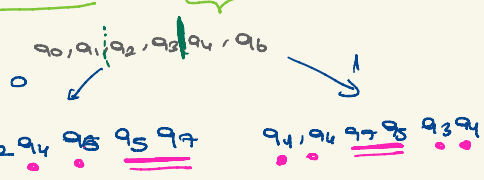
DFA indirgenmesi

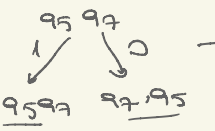
	x=0	x=1
→ q_0	q_0	q_1
q_1	q_2	q_4
q_2	q_4	q_7
q_3	q_5	q_5
q_4	q_5	q_3
q_5	q_5	q_7
q_6	q_7	q_5
q_7	q_7	q_5

* DFA indirgenmesinde ilk olarak üç durumlar ayrılır

$P_0 = (q_0, q_1, q_2, q_3, q_4, q_6)(q_5, q_7)$

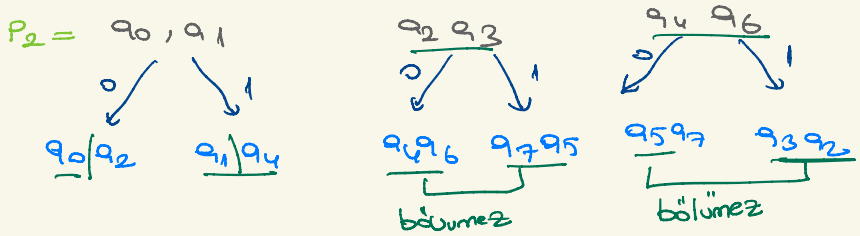
$P_1 =$





→ Sabit kolo adı daha fazla bölünemez.

$P_1 = (q_0, q_1)(q_2, q_3), (q_4, q_6)(q_5, q_7$



$P_2 = (q_0)(q_1)(q_2, q_3)(q_4 q_6)(q_5, q_7)$

$S_0 \quad S_1 \quad S_2 \quad S_3 \quad S_4$

Moore Makinesi İndirgenmesi

S	x=0	x=1	Z
A	C	B	0
B	B	D	1
C	A	H	2
D	E	G	1
E	C	D	2
F	C	H	2
G	G	H	1
H	F	B	1

* Z değerleri aynı olanlar bir grupta ayrı ayrı yazılır.

$P_0 = (A)(B,D,G,H)(C,E,F)$

$P_1 = B\ D\ GH$

0 → BE GF

1 → DGHB

CEF

0 → ACC

1 → HDH

$P_1 = (BG)(DH)(C)(EF)(A)$

$S_3 \quad S_4 \quad S_2 \quad S_5 \quad B_1$

* Partition işlemi ondan önceki partition'dan fazla ve daha fazla bölünemez.

S	x=0	x=1	
S_0	S_1	S_2	0
S_1	S_0	S_3	2
S_2	S_2	S_3	1
S_3	S_4	S_2	1
S_4	S_1	S_3	2

Sadeleşmiş girişlerin çıkışları alınırken çıkışını alırken tüm durumları bakılarak alınır.

$S_0 = A$

$S_1 = B$

$S_2 = BG$

$S_3 = DH$

$S_4 = EF$

Medy Makinesi İndirgenmesi

	x=0	x=1
→ A	A,0	D,1
B	C,0	E,1
C	G,0	E,1
D	G,0	F,1
★ E	E,1	C,0
F	B,0	D,1
G	B,0	E,1

$P_0 = (ABCDEFG)$

$P_1 = (E)(ABCDFG)$

$P_2 = (E)(ADF)(BCG)$

$P_3 = (A)(DF)(BCG)$

II. Düzgün Kümeler ve Düzgün Deyimler

Düzgün Kümeler

* Sonlu özdevinir (FA) modeli, giriş alfabesindeki simgelerden oluşan dizgilerin bir kısmını tanıyan, bir kısmını tanımayan modeldir.
* Her sonlu özdevinirin tanıdığı bir dizgi kümesi vardır. Sonlu özdevinir tarafından tanınan kümelere "düzgün kümeler (regular set)" denir.

- $P_1 = \{1001, 0011, 1100, 1010\}$
- P_2 111'in alt dizgi kümesi
- P_3 ilk ve son hanesi 1 olan beş bitlik dizgi

} Düzgün kümedir.

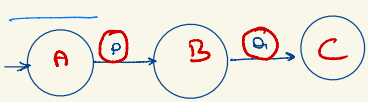
Düzgün Deyimler

* Düzgün Deyimler, düzgün kümeleri biçimsel olarak tanımlamak için kullanılan bir anlatım aracıdır. Her düzgün deyim bir alfabe simgesinden oluşturulan dizgilerin altkümesini tanımlar.

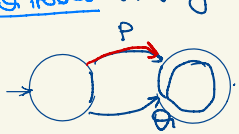
- $\{a, b, c, d, \ldots\ldots\ldots \Sigma\}$
- $\{a?, b?, \{c?\}, \{\Sigma\}, \{\}\}$

* P ve Q düzgün deyimse; P+Q, P*, Q*, P×Q bu ifadelerde düzgün deyimdir.

P ve Q
PQ ifadesi;

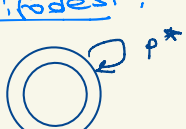


P+Q ifadesi (P veya Q)



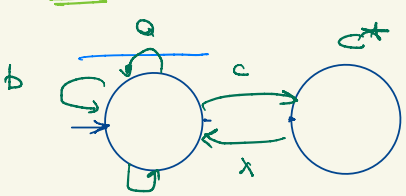
P* ifadesi;

↳ kendisi hiç gelmesede tanıyabilir. ve sonsuz defada gelse tanıyabilir.



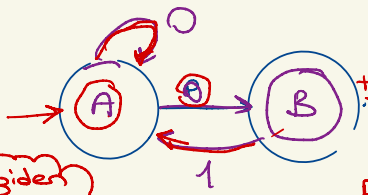
Ör1: $(a+b+c^*)^*$

✓ içten dışa doğru çalışır.



Not: $R = \Theta + RP \rightarrow R = \Theta P^*$ → * Sonlu durumu tanıyan düzenli ifadeyi tanımlamak için kullanılır.

Örn:



$A = \lambda + A0 + B1$

$B = A0$

$A = \lambda + A0 + A01$

$A = \underbrace{\lambda}_{\Theta} + \underbrace{A}_{R}\underbrace{(0+01)}_{P}$

$A = 0.(0+01)^*$

⇓

Sonlu Durum ifadesinin denkleni

* A'ya gider okları boş bırak

* B'ye giden okları boş bırak

* B gördüğün yere A0 yaz.

* A'ya gelince de gelebiliyoruz 01 gelince de B den A ya geliyoruz oyüzden (0+01) ifadesine P diyoruz.

III. Dil Bilgisi ve Diller

Dil Bilgisi ve Diller

* Her biçimsel dil belirli bir alfabe üzerinde tanımlanır. Alfabe ise sonlu simgeden oluşan bir kümedir.

* A, B, C, D, ... → Söz dizimi değişkeni

* a, b, c, d, ... 0, 1, 2, ... → Uç simge

* $\alpha, \beta, \gamma, \ldots\ldots$ → tümsel yapı

* $G = \langle V_N, V_T, P, S \rangle$

V_N = Söz dizimi değişkeni

V_T = Uç simgeyi

P = Üretme kurolidir.

S = Start(başlangıç değeri) $[S \in V_N]$

* $\alpha \Rightarrow \beta$

* $V = V_N \cup V_T$ ise $\alpha \in V^+$ ve $\beta \in V^*$ (V^+ → bir kere, V^* → yo hiç)

$V^+ = V^* - \{\lambda\}$ (Bir kere ve sonsuz defa gelmesi durumu. Yo hiç defa gelmesi durumunun çıkarılmış halidir.)

* $L(G)$ şeklinde tanımlanır.

$L(G) = \{w \mid w \in V_T^+, S \overset{*}{\Rightarrow} w\}$ → Dilin Tanımı

- $G = \langle V_N, V_T, P, S \rangle$

$V_N = \{S\}$ P: $S \rightarrow 0S1$, $S \rightarrow 01$

$V_T = \{0, 1\}$

* $S \rightarrow 0S1 \rightarrow 00S11 \rightarrow 000111$ (3 tane 0, 3 tane 1)

$S \rightarrow 01 \rightarrow$ 1 tane 0, 1 tane 1

$\rightarrow L(G) = \{0^n 1^n \mid n \geq 1\}$

Uyarı !

* $0^n 1^n$ ifadeki n ifadesi üst ifadesi değildir. n kaç kez tekrar etiğini gösterir.

Dillerin Sınıflandırılması

* Dil bilgisi ve üretimleri dilleri, yeniden yazma kurallarına göre üç ayrılır

1) Tür-0 ⇒ Kısıtlamasız
2) Tür-1 ⇒ Bağlama Bağımlı
3) Tür-2 ⇒ Bağlamdan Bağımsız
4) Tür-3 ⇒ Düzenli ve Düzgün

Tür-0 Dil Bilgisi (Recursively Enumerable)

* $\alpha \Rightarrow \beta : \alpha \in V^+$, $\beta \in V^+$

* $G = \langle V_N, V_T, P, S \rangle$

- $V_N = \{S, L, R, A, B, C\}$
- $V_T = \{a\}$
- P = S → LAaR
  Aa → aaA
  aA → aaA
  LB → LA
  aC → Ca
  LC → λ
  AR → BR | C

S → LAaR → LaaRR → LaaC → LaCa → LCaa → aa

→ 2^n tane a gelecek, → a^n

S → LAaR → LaaAR → LaaBR → LaBaR ⇒ LBaaR → LAaaR → LaaAaR

→ LaaaAR → LaaaC → LaaCa → LaaCaa → LaCaaa → LCaaaa ⇒ aaaa

L_{a^n}

- $L(G) = \{a^{2^n} \mid n \geq 1\}$

Tür-1 Dil Bilgisi (Bağlama Bağımlı) $|\alpha| \leq |\beta|$

* $\alpha \Rightarrow \beta$ $\alpha \in V^+$ $\beta \in V^+$

* $G = \langle V_N, V_T, P, S \rangle$

- $V_N = \{S, A, B\}$
- $V_T = \{a, b, c\}$
- P = S = aSAB
  S = aAB
  BA = AB
  aA = ab
  bA = bb
  bB = bc
  cB = cc
- $L(G) = \{a^n, b^n, c^n \mid n \geq 1\}$

Türet: aAB → abB → abc

Türet: aSAB → aaABAB → aaBABB → aabABB → aabbBB → aabbcB → aabbcc

- "Önce a'dan n tane sonra b'den n tane sonra c'den n tane gelecek."

Tür-2 Dil Bilgisi

* $A \rightarrow \beta : A \in V_N$ $\beta \in V^*$

- $G = \langle V_N, V_T, P, S \rangle$
  $V_N = \{S, A, B\}$
  $V_T = \{a, b\}$
  P = S = aB | bA
  A = a | aS | bAA
  B = bS | b | aBB

S ⇒ bA ⇒ baS ⇒ babA ⇒ baba

S ⇒ aB ⇒ abS ⇒ abbA ⇒ abbaS ⇒ abbaaB ⇒ abbaab

S ⇒ bA ⇒ bbAA ⇒ bbaSA ⇒ bbaaBA ⇒ bbaabSA

⇒ bbaabaBA ⇒ bbaababA ⇒ bbaababa

Tur-3 Dil Bilgisi

* A → aB

A → a

A → λ

A, B ∈ V_N , a ∈ V_T

rnek 3.5.

$G_{3.5} = < V_N, V_T, P, S >$

$V_N = \{S, A, B\}$

$V_T = \{0, 1\}$

P: $S \Rightarrow 0S \mid 0A \mid 0 \mid \lambda$

$A \Rightarrow 0B$

$B \Rightarrow 1S$

$S \Rightarrow 0S \Rightarrow 00$

$S \Rightarrow 0A \Rightarrow 00B \Rightarrow 001S \Rightarrow 001$

$S \Rightarrow 0S \Rightarrow 00A \Rightarrow 000B \Rightarrow 0001S \Rightarrow 00010A$

$\Rightarrow 000100B \Rightarrow 0001001S \Rightarrow 00010010$

Sağ Doğrusal Dil Bilgisinin Turetildigi Dili Tanıyıcı

Özdevinirin Bulunması

Bir sağ-doğrusal dilbilgisi (G) verildiğinde, T(M) = L(G) eşitliğini sağlayan sonlu özdevinir (M) aşağıdaki gibi bulunur.

$G = < V_N, V_T, P, S >$

$M = < Q, \Sigma, q_0, \delta, F >$

$Q = \{ [S], [\lambda], [\alpha_1], [\alpha_2], [\alpha_3], \ldots \}$

* L(G) sağ doğrusal :

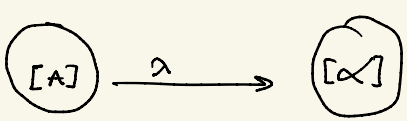
$G = \langle V_N, V_T, P, S \rangle$

$M = \langle Q, \Sigma, \delta, q_0, F \rangle$

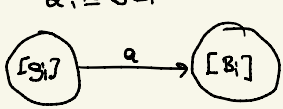
α_i = suffix , son ek

* $Q = \{ [S], [\lambda], [\alpha_1] [\alpha_2] \ldots \ldots \}$

- $F = [\lambda]$
- $q_0 = [S]$
- $\Sigma = V_T$
- $S: A \rightarrow \alpha$



$\alpha_i = aB_i$



→ $G = \langle V_N, V_T, P, S \rangle$

$V_N = \{S, A, B, C\}$

$V_T = \{0, 1\}$

$P = S = 0S \mid 1S \mid 11A$

$A = 0A \mid 1A \mid \lambda$

$Q = \{[S], [0S], [0A], [1A], [1], [11A], [11A], [A]\}$

$\Sigma = \{0, 1\}$ * harfleri giriş simgesi olarak alıyor

$F = \{\lambda\}$

$q_0 = \{S\}$

$(QA, 0) = A$

$111A, 1 = 11A$ (geçme)

$\delta = s([S], \lambda) = [0S]$

$s([S], \lambda) = [1S]$

$s([S], \lambda) = [11A]$

$s([1], \lambda) = [0A]$

$s([1A], \lambda) = [1A]$

$s([A], \lambda) = [\lambda]$

$s([0S], 0) = [S]$

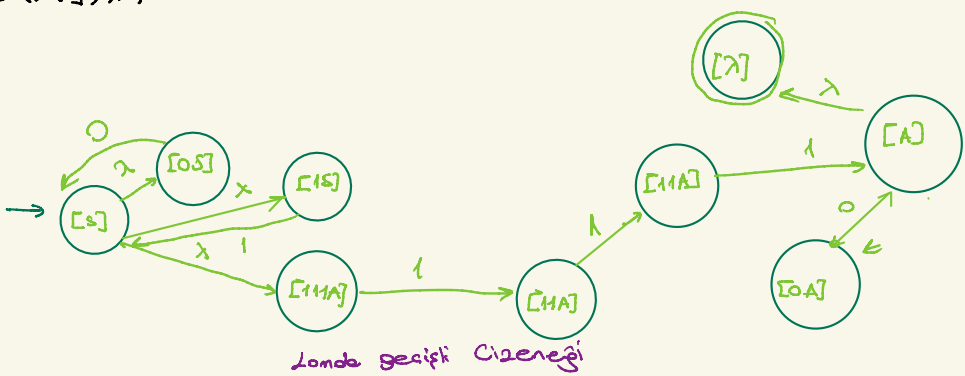
$s([1S], 1) = [S]$

$s([11A], 1) = [11A]$

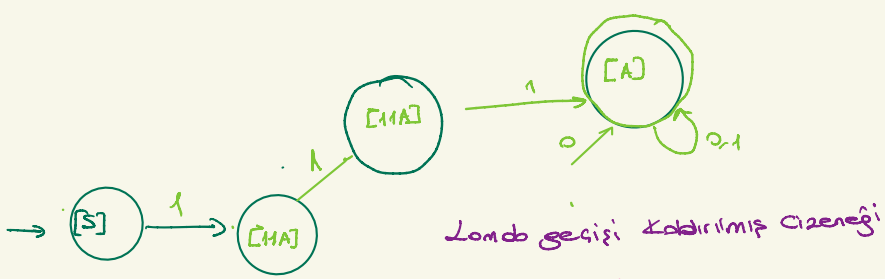
$s([11A], 1) = [1A]$

$s([1A], 1) = [A]$

$s([0A], 0) = [A]$



Lamda geçişli Çizeneği



Lamda geçişi kaldırılmış Çizeneği

Sol Doğrusal Bil Bilgisi Türetiği Dil Tanıyan Sonlu Özdevinir.

Özdevinirin Bulunması

Sol-dorusal bir dilbilgisi (G) verildiğinde, bu dilbilgisinin türettiği dili tanıyan makine (M) aşağıdaki gibi bulunur.

1. Dilbilgisindeki tüm yeniden yazma kurallarının sağ tarafları tersine çevrilerek G^R elde edilir. G^R sağ-doğrusal bir dilbilgisidir.
2. G^R'yi tanıyan makine (M^R) bulunur.
3. M^R tersine çevrilerek M elde edilir. Bunun için :
   - Başlangıç durumu uç durum, uç durumlar ise başlangıç durumu yapılır.
   - Tüm durum geçişleri tersine çevrilir.

- $G = \langle V_N, V_T, P, S \rangle$

$P = S \to 0|S|S0$

$P^r = S|0|0S|S$

$V_N = \{S\}$

$V_T = \{0,1\}$

- $\Theta^r = \{[S], [\lambda], [0S1], [1S], [0]\}$

$\Sigma = \{0,1\}$ $\quad F^R = \{\lambda\}$

$q_0^R = \{S\}$

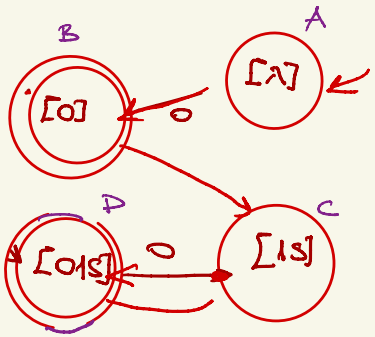
$\delta^R = \{[S], \lambda\} = [0]$

$\delta^R = \{[S], \lambda\} = [0|S]$

$\delta^R = \{[0S], 0\} = [1S]$

$\delta^R = \{[1S], 1\} = ([S])$

$\delta^R = \{[0], 0\} = ([\lambda])$



Sonlu Özdevinirin Tanıdığı Dili Türeten Düzgün Dil Bilgisinin Bulunması

❗ Sonlu özdevinir DFA olmak zorundadır.

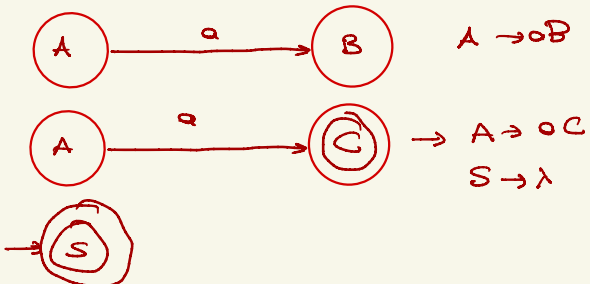
✱ $G = \langle V_N, V_T, P, S \rangle$

$V_N = Q$

$V_T = \Sigma$

$S = q_0$

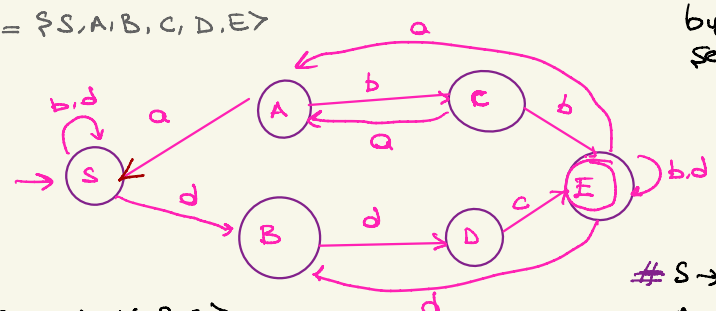
S:



- $M = \langle Q, \Sigma, q_0, F, S \rangle$

$Q = \{S, A, B, C, D, E\}$

✱ Önce küçük olan harf sonra büyük olan harf yazılacak şekilde işlem yapılır.



→ $G = \langle V_N, V_T, P, S \rangle$

$V_N = \{S, A, B, C, D, E\}$

$V_T = \{a, b, c, d\}$

$S \to aA|bS|dS|dB$

$A \to bC$

$B \to dD$

$C \to bE|aA|b$

$D \to CE|c$

$E \to aA|dB|bE|b|dE|d$

Pumping Lemma

* L uzunluğundan daha string p den büyük veya eşit olan herhangi bir s dizisi (x, y, z) 3e bölerek;

$y > 0$ olmalı

xy uzunluğu p den küçük veya eşit olma

$i \geq 0$ $(xy^i z \in L)$ ise bu dil düzenlidir.

● $L = \{0^n 1^n \mid n \geq 0\}$ bu dilin düzenli olup olmadığını bulunuz.

p = 3 için; 000 111

• xyz xy ilk p karakter içinde olmalı

000111

xy

x = 0

y = 00

z = 111

$xy^0z = 0111$

$xy^1z = 000111$

$xy^2z = 00000111$

⚠ Bu dil düzgün bir dil değildir.

IV. CFG – Tür-2

CFG- Tür-2 (Bağlamdan Bağımsız Diller)

* V_N, V_T, P, S

$\hookrightarrow A \Rightarrow B$

$V_N \quad V^* \quad (V_N \cup V_T)^*$

● $V_N = \{S\}$

$V_T = \{+, -, *, 1, (,), V, C\}$

$P = S \to S+S \mid S-S \mid S* \mid S/S \mid (S) \mid V \mid C$

Türetilebilecek Tümceler ⇒ ● S+S+S → S+S*S

● S+(S)*S

● S+(S/S)*S

● V+(V/C)*V

● $V_N = \{S, A, B\}$

$V_T = \{a, b, c\}$

$P = S \Rightarrow aAdd$

$A \Rightarrow aAd \mid Ad \mid bBcc$

$B \Rightarrow bBc \mid Bc \mid \lambda$

Türetme: ● S → aAdd → aAddd

→ abBccddd → obccddd

→ abBccddd

↪ obccddd

* $L(G) = \{a^n, b^m, c^k, d^p \mid n, m, k, l \geq 1, p > n, k > m\}$

* Aynı kural da olan üç simgelere bakıcaz.

Türetme Ağaçları (Parsing Tree)

* Bir dil bilgisi tarafından türetilen tümcesel yapı ve aşağıdaki gibi gösterilir:

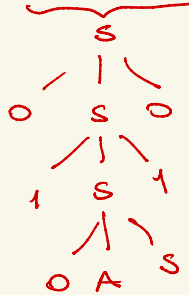
$S \Rightarrow \beta_1 \Rightarrow \beta_2 \Rightarrow \dots\dots\dots \Rightarrow \beta_{n-1} \Rightarrow \beta_n \Rightarrow w \qquad \beta_i \in V^*, w \in V_T^+$

* Bağlamadan-bağımsız dillerin her tümcesel yapısına ve her tümcesine bir türetme ya da ayrıştırma ağacı (derivation or parsing tree) karşı gelir.

- $\beta = 010A010$ Tümcesel yapısı ile

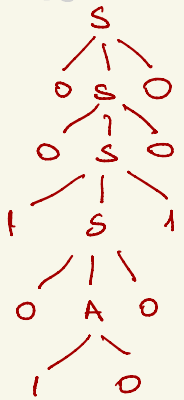
$w = 00101001010$ Tümcesine

- $\beta = 010A010$



$S \rightarrow 0S0 \mid 1S1 \mid 0AS$

- $w = 00101001010$



$S \rightarrow 0S0 \mid 1S1 \mid 0A0$

$A \rightarrow 1 \mid 0$

* Türetmeyi soldan başlayarak aşağı doğru ardından yukarıya doğru türetilir.

Türetme ya da Ayrıştırma Ağacının Tanımı

a) Ağacın kökünün etiketi S'dir.

b) Kök dışındaki ara düğümlerin etiketleri sözdizim değişkenleridir ($A \in V_N$)

c) Eğer ağaç bir tümcesel yapıya karşı geliyorsa, yaprakların etiketleri sözdizim değişkenleri ya da uç simgeler olabilir ($X \in V$). Eğer ağaç bir tümceye karşı geliyorsa yaprakların etiketleri yalnız uç simgeler ($a \in V_T$) olabilir.

d) Eğer bir ara düğümün etiketi A, bu ara düğümün hemen altındaki düğümlerin etiketleri ise soldan sağa $X_1, X_2, X_3, \dots, X_k$ ise, dilbilgisinin yeniden yazma kuralları arasında

$A \Rightarrow X_1 X_2 X_3 \dots X_k \qquad X_1, X_2, X_3, \dots, X_k \in V$

kuralı yer almalıdır.

e) Eğer bir düğümün etiketi λ ise, bu düğüm bir uç düğüm (yaprak) olmalı ve bu düğümün kardeşi bulunmamalıdır.

Soldan Sağdan Türetme

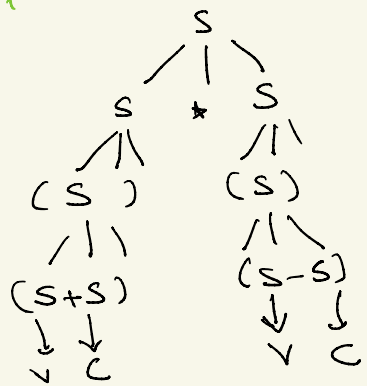
Yapraklarının etiketleri uç simgeler olan her türetme ağacına dilin bir tümcesi karşı gelir. Eğer dil belirgin (*unambiguous*) bir dil ise de, dilin her tümcesine bir türetme ağacı karşı gelir. Eğer dilin bir tümcesine birden çok türetme ağacı karşı geliyorsa, dil belirgin olmayan (*ambiguous*) bir dildir. Belirginlik dilbilgisinin ve dilin en temel özelliklerinden biridir. Tümceleri birden çok anlam taşıyan belirgin olmayan dillerin uygulamada hiçbir değeri yoktur. Bu nedenle, kullanılan dillerin tümünün belirginlik özelliğini sağladığı varsayılacaktır. Buna göre dilin her tümcesine bir türetme ağacı, yapraklarının etiketleri uç simgeler olan her türetme ağacına da bir tümce karşı geldiğini söyleyebiliriz.

Dilin her tümcesine bir türetme ağacı karşı geldiği gibi, bir soldan türetme, bir de sağdan türetme karşı gelir.

Eğer bir tümce türetilirken, her adımda en soldaki değişkene bir türetme uygulanıyorsa, yapılan türetmeye soldan türetme (*leftmost derivation*) denir. Soldan türetmenin her ara adımında, eğer tümcesel yapı birden çok değişken içeriyorsa, öncelik en soldaki değişkene verilir.

Eğer bir tümce türetilirken, her adımda en sağdaki değişkene bir türetme uygulanıyorsa, yapılan türetmeye sağdan türetme (*rightmost derivation*) denir. Sağdan türetmenin her ara adımında, eğer tümcesel yapı birden çok değişken içeriyorsa, öncelik en sağdaki değişkene verilir.

● $w_1 = (v+c)^*(v-c)$



● Soldan Türetme (En soldan türeteceğiz)

* $S \to S^*S \to (S)^*S \to (S+S)^*S \to (v+S)^*S$

$\to (v+c)^*S \to (v+c)^*(S) \to (v+c)^*(S-S)$

$\to (v+c)^*(v-S) \to (v+c)^*(v+c)$

* Tek yapı olursa işleme girerler. Teker teker hesaplama yapılır.
* Terminallere gelene kadar bu işlem devam eder

● Sağdan Türetme (En sağdan türeteceğiz)

* $S \to S^*S \to S^*(S) \to S^*(S-S) \to S^*(S-c) \to S^*(v-c) \to$

$\to S^*(v-c) \to (S)\cdot(v-c) \to (S+S)^*\cdot(v-c) \to (S+c)(v-c) \to$

$\to (v+c)^*(v-c)$

* Sağdan ve soldan türetmede aynı ifade denk gelir.

Dil Bilgisin Sadeleştirilmesi

* Bağlamdan-bağımsız bir dil bilgisi verildiğinde, bu dil bilgisinin daha az sayıda değişken ve kural içeren, ve belirli biçimlerdeki kuralları içermeyen bir dil bilgisine dönüştürülmesine "dil bilgisin sadeleştirilmesi" denir.

Özyineli Kural / Değişken

* Bir yeniden yeniden yazma solundaki değişken, kuralın sağında yer alıyorsa bu kurala "özyenili kural" denir.

* Eğer bir yeniden yazma kuralının solundaki değişken, aynı zamanda kuralın sağındaki ilk simge ise, başka bir deyişle kural $(A \Rightarrow A\alpha_2)$ biçiminde ise, bu kurala **doğrudan özyineli kural** (*direct recursive rule*) denir.

Yok edilebilir Değişken

* Eğer bir değişkenin yerine bir ya da birkaç türetme sonunda λ konulabiliyorsa,

$A \Rightarrow \lambda$ ya da $A \overset{*}{\Rightarrow} \lambda$

bu değişkene yok edilebilir (*nullable*) değişken denir. Bazı işlemlerde, başlangıç değişkeni dışında dilbilgisinde yok edilebilir değişken bulunmaması; başka bir deyişle $(S \Rightarrow \lambda)$ dışında λ-kuralının bulunmaması istenir. Buna göre, eğer dil λ'yı içermiyorsa $(\lambda \notin L)$, dilbilgisinde hiç λ-kuralı bulunmayacaktır. Eğer dil λ'yı içeriyorsa da $(\lambda \in L)$, dilbilgisindeki tek λ-kuralı $(S \Rightarrow \lambda)$ olacaktır.

* Bir algoritmayla, dilbilgisinin yok edilebilir değişkenlerinin hangileri olduğu kolaylıkla bulunabilir. Bir başka algoritmayla da dilbilgisi, başlangıç değişkeni dışında yok edilebilir değişkeni bulunmayan eşdeğer bir dilbilgisine dönüştürülebilir. Bu algoritmalar aşağıda verilmektedir.

● $B \to bBCC \mid \lambda$

$B \to \lambda$ veya $B^* \to \lambda$

* Eğer, CFG λ içermiyor hiç λ olmaz,
* Eğer, CFG, λ içeriyorsa $S \to \lambda$ olmalıdır.

Yok Edilebilen değişkenin Bulunması

1) $T_L = \{A \mid A \Rightarrow^* \lambda$ GP$\}$
2) Teski $= \emptyset$
3) $T_L =$ Teski Olana Kadar
   - 3.1 = Teski $= T_L$
   - 3.2 = V_N deki $\forall A$ için; Eğer en az bir $A \to \alpha$ için $\alpha \in [T_L]^*$ için A'yı T_L ekle

● $V_N = \{S, A, B, C\}$ P = $S \to ACA$, $A \to B \mid C \mid \alpha A \alpha$, $B \to bB \mid b$, $C \to cC \mid \lambda$

$V_T = \{\alpha, b\}$

● Boşlamda değişkeni dışında yok edilebilir değişkeni bulunmayan eş değer dil bilgisini bulunuz?

Çözüm:

1) $T_L = \{C \mid C \to \lambda\}$ $T_L = \{C\}$
2) Teski $= \emptyset$
3) $T_L =$ Teski olmadığı sürece

Teski $= \{C\}$

1) ● C'ye getiren söz dizim değişkeni $\{A\}$

$T_L = \{A, C\}$

2) $T_L = \{A, C\}$ Teski $= \{C\}$ → Dolaylı yoldan C'ye gelebilme durumuna bakılır.

● $T_L = \{S, A, C\}$

Yok edilebilir değişken olabilir Amo yok edilirse dilde yok edilmiş olur.

→ Sildik

$S \to ACA$ →
$A \to B$
$A \to C$
$A \to \alpha A \alpha$
$B \to bB$
$B \to b$
$C \to CC$
$S \to \lambda$
$C \to \lambda$

$S \to \lambda \mid A \mid \in [A] \mid AC \mid CA \mid AA \mid ACA$
$A \to B \mid C \mid \alpha \mid \alpha A \mid \alpha A \alpha \mid \alpha\alpha$
$B \to bB \mid b$
$C \to CC \mid c$

Başlangıç Değişkeni (S) yok edilebilir Değişkeni Bulunmayan Eş değer Dil Bilgisi

1) P'deki tüm $A \to \lambda$ Kurolarını çıkar.
2) Eğer $\lambda \in L(G)$ P'ye $S \Rightarrow \lambda$ ekle
3) $\forall A \to \alpha$ Eğer α yok edilebilir değişken içeriyorsa bu değişkenlerin bir yada bir kaçının α'dan çıkarılarak oluşabilecek her α_i için P'ye $A \Rightarrow \alpha_i$ ekle

* A burda tüm soldaki değişkeni ifade eder.

Birim Türetme Kuralları

* Dil bilgisindeki yeniden yazma kurallarından biçiminde olanlara "birim türetme kuralı" yada "zincir kural" denir. Birim türetme kuralları tümdeleri türetilmesi geciktiren kurallardır.

Her Değişkenden üretilebilecek Değişkenlerin Kümesi

- $\forall A \in N_N = T_A = \{B \mid B \in V_N \quad A^* \Rightarrow B\}$

I. $T_A = A$

II. $T_{eski} = \emptyset$

III. $T_A = T_{eski}$ olduğuna kadar $T_{yeni} = T_A - T_{eski}$, $T_{eski} = T_A$

T_{yeni}'deki $\forall B$ için $T_A = \{T_A \cup \{C?\}$

Birim Türetme Kuralı içermeyen Dil Bilgisinin Bulunması

I. $P = P - \{A \Rightarrow B \mid A, B \in V_N\}$

II. V_N'deki $\forall A$ için T_A'dan A'dan başka sağ2 dizimi değişkeni varsa

$T_A = T_A - \{A\}$

T_A'daki $\forall B$ için $B \to \beta_1 \mid \beta_2 \mid \dots \beta_N$ ise

$P = P \cup \{A \to \beta_1 \mid \beta_2 \mid \beta_3 \mid \dots \mid \beta_N\}$ yazılır.

Örn: ● $S \to A \mid AA \mid AC \mid CA \mid ACA \mid \lambda$ → S'e A, B ve C durumları yaz
$A \to B \mid oAo \mid oo$ → A'ya B deki durumları yaz
$B \to c \mid bB \mid b$ → B'ye C durumları
$C \to cC \mid c$ → C'ye C

Çözüm: $T_S = \{S, A, B, C\}$
$T_A = \{A, B, C\}$
$T_B = \{B, C\}$
$T_C = \{C\}$

S → AA | AC | CA | ACA | λ | oAo | oo | b | bB
A → oAo | oo | bB | b
B → bB | b | cC | c
C → cC | c

Tekil olarak A, B, C, S yazmayarak devam edilir

Yığınlı Özdevinirler (PDA)

* Yığınlı özdevinirler, bir CFG (Tür-2) tarafından türetilen CFG'yi tanıyan makine modelidir.

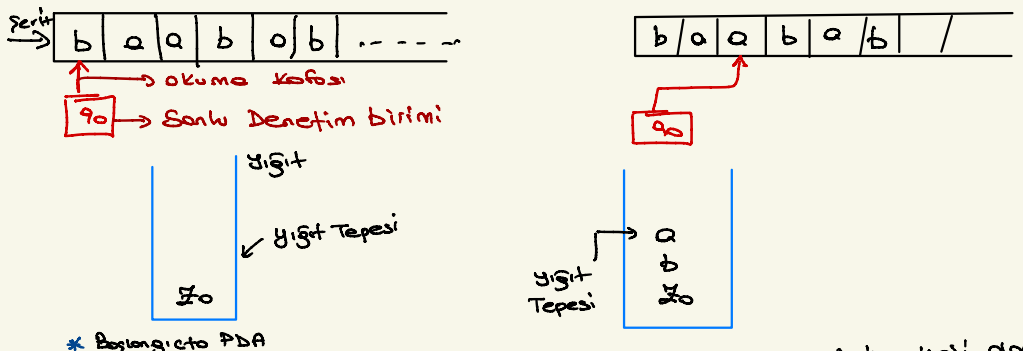
Yığınlı Özdevinirin Temel modeli

* Biçimsel olarak PDA bir 7li olarak tanımlanır.

$PDA = \langle Q, \Sigma, \Gamma, q_0, Z_0, F \rangle$

- # Q = Sonlu sayıda durum içeren durumlar kümesi
- # Σ = Giriş alfabesi
- # Γ = Sonlu sayıda simge içeren yığıntı alfabesi
- # q_0 = Başlangıç Durumu
- # Z_0 = yığıntılı simgelerden, yığıntı başlangıç simgesi olarak özel bir simgedir.
  - Not = * Z_0 giriş alfabesinde olmaz. ($Z_0 \in \Gamma$)
- # F = Uç durum kümesi
- # δ = Durum geçiş işlevi

* Deterministik PDA $[Q \times (\Sigma \cup \{\lambda\}) \times \Gamma)$'dan $[Q \times \Gamma^*]$'a ve Non deterministik PDA $[Q \times (\Sigma \cup \{\lambda\}) \times \Gamma)$'dan $[Q \times \Gamma^*]$'a en sonlu alt kümelerini işler.



* Başlangıçta PDA

* Makinenin iki türlü Hareketi vardır. Normal hareket ve λ hareketi olarak adlandırılır.

Normal Hareket

* Şerite bir simge okunur.
* Okuma kafası sağa geçer.
* Yığıntının en üsteki simge silinir. (pop).
* Yığıntının üstüne, yığıntı simgelerinden oluşan bir dizge ($\alpha \in \Gamma^*$) eklenir. (push)

→ Bu dizgide boş (λ) da olabilir.

Eğer dizgi $\alpha = x_1, x_2, x_3 \ldots\ldots x_n$ gibi çok yığıntı simgesinden oluşuyorsa en sonlaki (x_1) en üste gelecek şekilde yığıntıya eklenir.

λ hareketi

* Şeriden simge okunmaz, okuma kafası sabittir.
* Yığıntının en üstteki simgeye "pop" denir.
* yığıntının üstünde ($\alpha \in \Gamma^*$) yığıntı simgelerden oluşan diğer eklersin oluşan diziyi λ olabilir.

Anlık PDA Tanımımı

* PDA'nın belirli bir andan sonraki davranışları kesinleştirebilmek için o ana ilişkin 3 bilginin verilmesi gerekir.
  - PDA'nın (sonlu den. bir) bulunduğu durum
  - Giriş dizgisinin henüz işlenmemiş kesmi
  - Yığıntının içeri

$(ID) = (P, V, x)$

P = PDA'nın durumunu
V = Giriş Dizgisinin işlenmemiş Kesmini
x = yığıntının içeriğini

Örnek: $L = \{w c w^R \mid w \in (0+1)^*\}$ bu dili türeten dil bilgisi

$G = \langle V_N, V_T, P, S \rangle$

$V_N = \{S\}$

$V_T = \{0, 1, c\}$

$P = S = 0S0 \mid 1S1 \mid C$

- Bu dili tanımlayan PDA $M = \langle Q, \Sigma, \Gamma, q_0, \delta, z_0, \emptyset \rangle$

$Q = \{q_0, q_1\}$

$\Sigma = \{0, 1, c\}$

$\Gamma = \{0, 1, c, z_0\}$

* PDA'nın çalışma ilkesi = Bu PDA'nın tanıdığı dizgiler $\{0,1\}^*$ alfabesindeki herhangi bir w alt dizgisi ile bunun tersinden oluşturulmaktadır ve w ile w^R arasında "c" simgesi yer almaktadır. Bu dizgiyi tanıyan PDA;
  - → önce w'nin simgeleri okunur, okunan her simge yığıntıya eklenir. Ekleme sırasında PDA q_0 durumunda durur.
  - → c'yi okuduğu zaman w'nin bittiğini anlar, yığıntıda bir değişiklik yapmaz ve q_1 durumuna geçer.
  - → Sonra w^R nin simgelerini okur ve okuduğu her simge için eğer yığıntı tepesi aynı simge var ise bu simgeyi yığıntıdan siler (push)
  - → Giriş dizgisinin tüm simgeleri okunduktan sonra yığıntı tepesinde z_0 varsa bunu da siler ve yığıntıyı tamamen boşaltır.

$\delta = \delta(q_0, 0, Z_0) = (q_0, 0Z_0)$

$\delta(q_0, 1, Z_0) = (q_0, 1Z_0)$

$\delta(q_0, c, Z_0) = (q_1, Z_0)$

$\delta(q_0, 0, 0) = (q_0, 00)$

$\delta(q_0, 1, 0) = (q_0, 10)$

$\delta(q_0, 0, 1) = (q_0, 01)$

$\delta(q_0, 1, 1) = (q_0, 11)$

$\delta(q_0, c, 0) = (q_1, 0)$

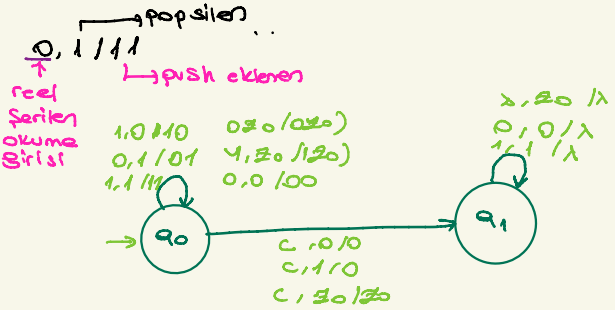
$\delta(q_1, c, 1) = (q_1, 1)$

$\delta(q_1, 0, 0) = (q_1, \delta)$

$\delta(q_1, \beta, 1) = (q_1, \lambda)$

$\delta(q_1, \lambda, Z_0) = (q_1, \lambda)$

* Geçişler birbirinden bağımsızdır. ve olabilecek tüm geçişlerdedir.



PDA'nın Deterministlik olma koşulu

* Her CFL bir PDA bulunur. Bulan her PDA deterministik değildir. Deterministik PDA için:

- Her $\delta(q_0, \varepsilon, X)$ için tanımlı en çok bir hareket olmalı
- $\delta(q, \lambda, X)$ için hareket tanımlı ise hiç bir giriş simgesi için $\delta(q, a, X)$ hareketin tanımlı olması gerekir.

PDA'nın q_0, q_1, q_2, q_3 ve q_4 durumları için tanımlı hareketler deterministik. Ancak q_3 için tanımlı olan hareketler deterministik değildir.

Bağlamdan Bağımsız (CFG) Dil bilgisi ve PDA Eş değeri

* Bir CFG'ye eşdeğer bir PDA varsa, bu PDA'ya eş değer CFG'de vardır.

Verilen bir CFG'nin Eş değer PDA'nın Bulunması

* Eğer G bağlamdan bağımsız bir dil bilgisi ise bu dil bilgisinin türettiği bağlamdan bağımsız dili $L(G)$ tanıyan bir PDA vardır.
* CFG G, PDA'yla gerçekleştirmek için verilen bir CFG'nin $T(M) = L(G)$ olan PDA iki yolda bulunur.

CFG Eş değer PDA'yı bulmak için 1. yöntem: (Boş yığıntıda tanıyan PDA) Bağlamdan bağımsız bir dil bilgisi $G = \langle V_N, V_T, P, S \rangle$ dir.

* Bu yöntem için verilen dil bilgisinin Greibach formda olması gerekmektedir.

$A \Rightarrow a\alpha$

$A \Rightarrow \lambda$: $A \in V_N$, $a \in V_T$, $\alpha \in V_N^*$

ÖRNEK: $G = \langle V_N, V_T, P, S \rangle$

$V_N = \{S\}$

$V_T = \{+, -, *, /, v, c\}$

$P: S \Rightarrow +SS \mid -SS \mid *SS \mid /SS \mid v \mid c$

Bu dilbilgisinin türettiği dili tanıyan PDA

$M = \langle Q, \Sigma, \Gamma, \delta, F, z_0, q_0 \rangle$

$Q = \{q_0\}$
$\Sigma = \{+, -, *, /, v, c\}$
$\Gamma = \{S\}$
$F = \phi$
$z_0 = S$
δ :

$\delta(q_0, +, S) = (q_0, SS)$
$\delta(q_0, -, S) = (q_0, SS)$
$\delta(q_0, *, S) = (q_0, SS)$
$\delta(q_0, /, S) = (q_0, SS)$
$\delta(q_0, v, S) = (q_0, \lambda)$
$\delta(q_0, c, S) = (q_0, \lambda)$

Dilbilgisinin türettiği örnek dizi..

$S \Rightarrow {}^*SS \Rightarrow {}^*+SSS \Rightarrow {}^*+vSS \Rightarrow {}^*+v/SSS \Rightarrow {}^*+v/vSS \Rightarrow {}^*+v/vcS \Rightarrow {}^*+v/vc-SS$

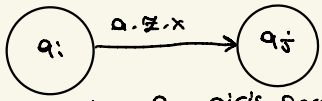
$\Rightarrow {}^*+v/vc-SS \Rightarrow {}^*+v/vc-vS \Rightarrow {}^*+v/vc-v*SS \Rightarrow {}^*+v/vc-v*cS \Rightarrow {}^*+v/vc-v*cv$

$w = (v+v/c)*(v-c*v)$

$(q_0, *+v/vc-v*cv, S) \vdash (q_0, +v/vc-v*cv, SS)$
$\vdash (q_0, v/vc-v*cv, SSS) \vdash (q_0, /vc-v*cv, SS) \vdash (q_0, vc-v*cv, SSS)$
$\vdash (q_0, c-v*cv, SS) \vdash (q_0, -v*cv, S) \vdash (q_0, v*cv, SS) \vdash (q_0, *cv, S)$
$\vdash (q_0, cv, SS) \vdash (q_0, v, S) \vdash (q_0, \lambda, \lambda)$

Yığıt boş oldu: okunan PDA w'yi tanır.

CFG'nin Eşdeğer PDA'nın bulunması (Üç Durum Tanıyan PDA)



$a \in V_T \cup \{\lambda\}$ $\quad Z \subset \Gamma$ $\quad \alpha \in \Gamma^*$

* Bu modelde a = giriş şeridinde okunan simge
Z = yığıntı tepesindeki simge
α = yığıntı eklenen yığıntı simgelerinin dil bilgisi

→ $\delta(q_i, a, \lambda) = (q_j, \alpha)$ yerine $\forall Z_k \in \Gamma \quad \forall (q_i, a, Z_k) = (q_j, \alpha, Z_k)$

- $Q = \{q_0, q_1, q_2\}$ $\quad \Gamma = V_N \cup V_T \cup \{Z_0\}$
$\Sigma = V_T$ $\quad F = \{q_2\}$
δ: $\delta(q_0, \lambda, \lambda) = (q_1, S)$ ve $\delta(q, \lambda, Z_0) = (q_2, \lambda)$ = Bağlamdan bağımsız olacak PDA Tanımlanır.
→ $\delta(q_1, \alpha, \alpha) = (q, \lambda)$ her $A \Rightarrow \alpha$ için PDA'da $\delta(q, \lambda, A) = (q, \alpha)$ Tanımlanır.

* Oluşturulan PDA üç durumdu tanır. ve aynı zamanda yığıntıyı boşaltır.

Örnek: $\{a,b\}$ alfabesinde

$G = \langle V_N, V_T, P, S \rangle$

$V_N = \{S\}$
$V_T = \{a, b\}$
P: $S \Rightarrow aSb \mid aSbb \mid ab \mid abb$ $\quad L = \{a^n b^m \mid 1 \le n, \; n \le m \le 2n\}$

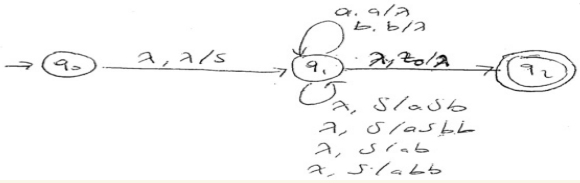
Verilen dilbilgisinde eşdeğer PDA:

$M = \langle Q, \Sigma, \Gamma, \delta, Z_0, q_0 \rangle$

$Q = \{q_0, q_1, q_2\}$
$\Sigma = \{a, b\}$
$\Gamma = \{a, b, S, Z_0\}$
$F = \{q_2\}$

δ:
$\delta(q_0, \lambda, \lambda) = (q_1, S)$
$\delta(q_1, a, a) = (q_1, \lambda)$
$\delta(q_1, b, b) = (q_1, \lambda)$
$\delta(q_1, \lambda, S) = (q_1, aSb)$
$\delta(q_1, \lambda, S) = (q_1, aSbb)$
$\delta(q_1, \lambda, S) = (q_1, ab)$
$\delta(q_1, \lambda, S) = (q_1, abb)$
$\delta(q_1, \lambda, Z_0) = (q_2, \lambda)$

PDA Non-det. dir.



Verilen bir PDA'dan Eşdeğer CFG'nin bulunması

* Eğer M bir PDA ise, türettiği dil bu PDA'nın $G(L) = T(M)$ bağlamdan-bağımsız bir dil bilgisi (CFG) vardır.

* $PDA = M = \langle Q, \Sigma, \Gamma, q_0, \delta, Z_0, F \rangle$
  - Verilen PDA'ya eşdeğer, Greibach bağlamdan bağımsız dili
    $G = \langle V_N, V_T, P, S \rangle$
    $V_T = \Sigma$
    $V_N = \{ S, [q, Ap] \mid A \in \Gamma, p, q \in Q \}$
  - P: PDA'nın her durumuna karşılık CFG'de
    $\forall q \in Q : S \Rightarrow [q_0, Z_0, q]$

- PDA'nın hareketine karşılık
  → $\delta(q_\varepsilon, a, A) = (q_J, B_2 \dots B_k) \in R$
  Her $p_1, p_2, \dots, p_k \in Q$ birleşimi için yeniden yazma kuralı
  $[q_i, A, p_\varepsilon) \Rightarrow a\,[q_J, B_1, p_i] \dots [p_{k-1}, B_k, p_i]$

Bu yöntemde verilen bir PDA'ya eş değer CFG:

1. Dil bilgisin uç simgeler kümesi PDA'nın giriş alfabesidir. $V_T = \Sigma$
2. Dil bilgisinin söz dizimi değişkenleri, S ile [durum, yığıntı simgesi, durum] üçlüsünden oluşur. Bu durumda PDA'da n adet durum ile k adet yığıntı simgesi varsa CFG de $kn^2 + 1$ söz dizimi değişkeni vardır.

Örnek: PDA da durumlar kümesi $\{q_0, q_1\}$ yığıntı alfabesi $\{A, B, Z_0\}$ ise

3. $(2)^2 + 1 = 13$ adet söz dizimi değişkeni oluşur.

$V_N = \{ S, [q_0, A, q_0], [q_0, A, q_1], [q_0, B, q_0], [q_0, B, q_1], [q_0, Z_0, q_0], [q_0, Z_0, q_1], [q_1, A, q_0], [q_1, A, q_1], [q_1, B, q_0], [q_1, B, q_1], [q_1, Z_0, q_0], [q_1, Z_0, q_1] \}$

3. CFG'nin yeniden yazma kuralları aşağıdaki gibi bulunur.
→ $\forall q \in Q : S \Rightarrow [q_0, Z_0, q]$
→ PDA'nın $\delta(q_0, \lambda, A) = (q_1, \lambda)$ biçimindeki hareketine karşılık $[q_0, A, q_1] \Rightarrow \lambda$ biçiminde yeniden yazma kuralı oluşturulur.
→ PDA'nın $\delta(q_0, \lambda, A) = (q_1, \lambda)$ biçimindeki hareketine karşılık $[q_0, A, q_1] \Rightarrow a$ biçiminde yeniden yazma kuralı oluşturulur.
→ PDA'nın $\delta(q_0, a, A) = (q_1, B_1)$ biçimindeki her hareketine karşılık, her durum için $(\forall q_1 \in Q)$ $[q_0, A, p_1] \Rightarrow a\,[q_1, B_1, p_1]$ biçimde yeniden türetme kuralı oluşturulur.
→ PDA'nın $\delta(q_0, a, A) = (q_1, B_1, B_2)$ biçimindeki hareketine karşılık, her p_1, p_2 durumu için $(\forall p_1, p_2 \in Q)$ $[q_0, A, p_2] \Rightarrow a\,[q_1, B_1, p_1][p_1, B_2, p_2]$ biçiminde yeniden yazma kuralı oluşturulur.

→ PDA'nın $\delta(q_0, a, A) = (q_1, B_1, B_2, B_3)$ $\forall p_1, p_2, p_3 \in Q$
$[q_0, A, p_3] \Rightarrow a\,[q_1, B_1, p_1][p_1, B_2, p_2][p_2, B_3, p_3]$

ÖRNEK: Aşağıdaki PDA verilsin.

$M = (Q, \Sigma, \Gamma, q_0, \delta, Z_0, \phi)$

$Q = \{q_0, q_1\}$
$\Sigma = \{0, 1\}$
$\Gamma = \{Z_0, X\}$

δ : $\delta(q_0, 0, Z_0) = (q_0, XZ_0)$
$\delta(q_0, 0, X) = (q_0, XX)$
$\delta(q_0, 1, X) = (q_1, \lambda)$
$\delta(q_1, 1, X) = (q_1, \lambda)$
$\delta(q_1, \lambda, Z_0) = (q_1, \lambda)$

Tanımlı dil $L = \{0^n 1^n \mid n \geq 1\}$

Bunu türeten CFG.

$G = (V_N, V_T, P, S)$ $\quad k=2 \quad n=2 \quad kn^2+1 = 2 \cdot (2)^2 + 1 = 9$

$V_T = \{0, 1\}$
$V_N = \{S, A_1, A_2, A_3, A_4, A_5, A_6, A_7, A_8\}$

$A_1 = [q_0, X, q_0]$ $\quad A_2 = [q_0, X, q_1]$ $\quad A_3 = [q_0, Z_0, q_0]$
$A_4 = [q_0, Z_0, q_1]$ $\quad A_5 = [q_1, X, q_0]$ $\quad A_6 = [q_1, X, q_1]$
$A_7 = [q_1, Z_0, q_0]$ $\quad A_8 = [q_1, Z_0, q_1]$

Yeniden yazma kuralları.

- PDA'nın durumlarına karşılık oluşturulan kurallar:

$S \Rightarrow [q_0, Z_0, q_0]$
$S \Rightarrow [q_0, Z_0, q_1]$

- $\delta(q_0, 0, Z_0) = (q_0, XZ_0)$ hareketine karşılık

$[q_0, Z_0, q_0] \Rightarrow 0[q_0, X, q_0][q_0, Z_0, q_0]$
$[q_0, Z_0, q_1] \Rightarrow 0[q_0, X, q_0][q_0, Z_0, q_1]$
$[q_0, Z_0, q_0] \Rightarrow 0[q_0, X, q_1][q_1, Z_0, q_0]$
$[q_0, Z_0, q_1] \Rightarrow 0[q_0, X, q_1][q_1, Z_0, q_1]$

- $\delta(q_0, 0, X) = (q_0, XX)$ hareketine karşılık

$[q_0, X, q_0] \Rightarrow 0[q_0, X, q_0][q_0, X, q_0]$
$[q_0, X, q_1] \Rightarrow 0[q_0, X, q_0][q_0, X, q_1]$
$[q_0, X, q_0] \Rightarrow 0[q_0, X, q_1][q_1, X, q_0]$
$[q_0, X, q_1] \Rightarrow 0[q_0, X, q_1][q_1, X, q_1]$

- $\delta(q_0, 1, X) = (q_1, \lambda)$

$[q_0, X, q_1] \Rightarrow 1$

- $\delta(q_1, 1, X) = (q_1, \lambda)$

$[q_1, X, q_1] \Rightarrow 1$

- $\delta(q_1, \lambda, Z_0) = (q_1, \lambda)$

$[q_1, Z_0, q_1] \Rightarrow \lambda$

P: $S \Rightarrow A_3 \mid A_4$
$A_3 \Rightarrow 0A_1A_3 \mid 0A_2A_7$
$A_4 \Rightarrow 0A_1A_4 \mid 0A_2A_8$
$A_1 \Rightarrow 0A_1A_1 \mid 0A_2A_5$
$A_2 \Rightarrow 0A_1A_2 \mid 0A_2A_6 \mid 1$
$A_6 \Rightarrow 1$
$A_8 \Rightarrow \lambda$

A_1, A_3, A_5, A_7 yoksuz değişkenlerdir.

P: $S \Rightarrow A_4$
$A_4 \Rightarrow 0A_2A_8$ $\quad \rightarrow$ Bu kurallar sadeleştirilirse $\Rightarrow$
$A_2 \Rightarrow 0A_2A_6 \mid 1$
$A_6 \Rightarrow 1$
$A_8 \Rightarrow \lambda$

$G = (V_N, V_T, P, S)$
$V_T = \{0, 1\}$ $\quad V_N = \{S, A\}$
P: $S \Rightarrow 0A_2$
$A \Rightarrow 0A \mid 1$

Turing makineleri

* Genel amaçlı bilgisayar için bir matematiksel modeldir. Herhangi bir makineyle yapabilecek işlemleri Turing makinesiyle modelemek mümkündür.

* Turing makinelerinin kullanım alanları aşağıdaki gibi sınıflandırılabilir.

Turing makinelerinin kullanım alanları aşağıdaki gibi sınıflandırılabilir:

1. **Dil tanıyıcı.** Turing makineleri kısıtlamasız (*unrestricted*) ya da özyineli sayılabilir (**r.e.** : *recursively enumerable*) dilleri tanımak için kullanılabilir. Buna göre Turing makinesi, verilen bir tümceninin dilin bir tümcesi olup olmadığını bulabilir.
2. **Hesaplayıcı.** Turing makineleri, kısmi özyineli tamsayı fonksiyonların (*partially recursive integer functions*) hesaplanmasında kullanılabilir.
3. **Dil üreticisi.** Turing makineleri ile, **r.e.** dillerin tümcelerini ardarda üretmek mümkündür. Eğer biçimsel bir dil bir Turing makinesi tarafından üretilebiliyorsa bu dil **r.e.** bir dildir. Diğer taraftan her **r.e.** dil için, dilin tüm tümcelerini aradarda üreten bir Turing makinesi vardır.

Bu kitapta Turing makinelerinin dil tanıyıcı ve hesaplayıcı olarak kullanımı üzerinde durulacak ve **r.e.** dilleri tanıyan ve basit hesaplamaları gerçekleştiren Turing makinesi örnekleri verilecektir.

* $M = \langle Q, \Sigma, \Gamma, \delta, q_0, B, F \rangle$
  - $B \in \Gamma$
  - $B \notin \Sigma$
  - B → Blank

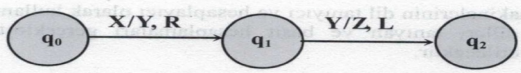
* Γ = Sonlu sayıda simge içeren şerit Alfabesi

[Q x Γ]'dan [Q x Γ x {L, R}]'ye

bir eşleme oluşturur. Bu tanımda yer alan **L** ve **R** simgeleri, hareketin sonunda okuma kafasının bir sağdaki (**R**) ya da bir soldaki (**L**) hücreye geçeceğini gösterir. Okuma kafasının hareketiyle ilgili seçenekler arasına "sağa/sola hareket etmeyip aynı hücre üzerinde kalma (**S**)" seçeneğini eklemek ve iki elemanlı {**L, R**} kümesi yerine üç seçenekli {**L, R, S**} kümesini kullanmak da mümkündür. Ancak bu kitapta çoğunlukla {**L, R**} kümesi kullanılacak ve Turing makineleri oluşturulurken, okuma kafasının her harekette bir sağ ya da bir sol hücreye geçmesi sağlanacaktır. Buna göre Turing makinesinin hareketleri aşağıdaki örneklerde görüldüğü gibi tanımlanacak, istenirse hareketler bir çizenekle de gösterilecektir.

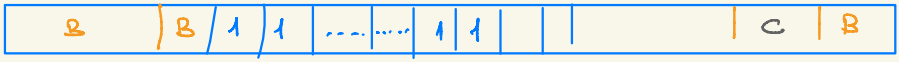
$\delta(q_0, X) = (q_1, Y, R)$ $\quad q_0, q_1, q_2 \in Q$

$\delta(q_1, Y) = (q_2, Z, L)$ $\quad X, Y, Z \in \Gamma$



* $T(M) = \{w \mid w \in V_T^+, (q_0, w) \overset{*}{\vdash} (\alpha_1, p, \alpha_2), \alpha_1, \alpha_2 \in \Gamma^*, p \in F$ şeklinde olursa makine w durumunu tanır. Komple şeridin tamamlanmasına gerek yoktur.

* P(n) = 2n Tanımlayan makine:



B 1 1 1 x B B B

Θ = {q0, q1, q2, q3, q4, q5, q6} # F = ∅

Σ = {1}

Γ = {B, 1, x}

Not: q_2 1) * Sağa ilerleyip, ilk B'nin yerine ?x? yaz ayırt edici

q_1 2) * Sola ilerle, ilk B'yi bul

q_2 3) * Sağa ilerle adım 1 değerini B yaz

q_3 4) * B'yi bulana kadar B yerine 1 yaz.

q_4 5) * Sağa ilerle B'yerine 1 yaz

q_5 6) okuma kafası şerile x'i okuyunca x'in solunda 1 bitince dur.

Döngüye girecek

	1	X	B
q_0	$(q_0, 1, R)$	—	(q_1, X, L)
q_1	$(q_1, 1, L)$	(q_1, X, L)	(q_2, B, R)
q_2	(q_3, B, R)	(q_5, B, R)	—
q_3	$(q_3, 1, R)$	$(q_2, X, R$	$(q_4, 1, R)$
q_4	—	—	$(q_1, 1, L)$
q_5	—	—	—

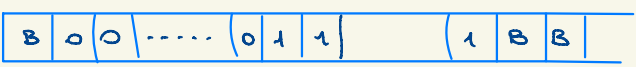
Durumların anlamları:

- q_0 : Başlangıç durumu. 1'ler öbeğinin sağına işaretleyicinin {ç} konulduğu durum bu durumdur.
- q_1 : Sola ilerleyip ilk B'nin bulunduğu durum.
- q_2 : 1'ler öbeğinin en solundaki 1'in silindiği durum.
- q_3 : Sağa ilerleyip ilk B'nin bulunduğu ve yerine 1 yazıldığı durum.
- q_4 : İkinci B'nin yerine 1 yazıldığı durum.
- q_5 : Son durum. Hesaplama tamamlandığında makine bu duruma gelir ve durur.

* Hesaplayıcı ilk konumlarda durur.

x y
000 111
q_2, 0, L
q_2, 0, R

● $L = \{0^n 1^n \mid n \geq 1\}$



1) Soldaki 0 yerine x yaz.
2) Sağa doğru ilerle, 1 yerine y yaz. Eğer birinci adımda silinen 0 karşılık 1 yoksa dur.
3) 1 ve 2 durumu tekrarla.
4) Eğer 1 kalmamışsa üç duruma geç. Eğer 1 kalmışsa dur.

q_0 = En soldaki sıfırı x yap

q_1 = Sağa ilerle 1'i y yap

q_2 = Sola ilerle, x'i bul, sağa geç

q_3 = Sağda 1 var mı?

q_4 = Üç. Durum

	0	1	X	Y	B
→ q_0	(q_1, X, R)	—	—	(q_3, Y, R)	—
q_1	$(q_1, 0, R)$	(q_2, Y, L)	—	(q_1, Y, R)	—
q_2	$(q_2, 0, L)$	—	(q_0, R, L)	(q_2, Y, L)	—
q_3	—	—	—	(q_3, Y, R)	(q_4, B, L)
(q_4)	—	—	—	—	—

* Dil Tanımlayıcı Tanımla Son durumda biter

6.3. Turing Makinesi Modelinde Değişiklikler

Turing makinelerinin temel modelinde yalnız bir ucu sonsuz, tek izli bir şerit ile tek okuma kafası kullanılmakta ve her harekette yalnız bir simge okunup-yazılabilmektedir (bkz 6.1). Şeridin iki yönde sonsuz olması, şerit sayısı, iz sayısı ve okuma kafası sayısı ile ilgili olarak Turing makinesi modelinde bir dizi değişiklik yapılabilir. Bu değişiklikler modelin kullanımında esneklik ve kolaylıklar sağlar. Ancak modelin gücünde hiçbir değişiklik yapmaz. Başka bir deyişle, hangi değişik model kullanılırsa kullanılsın, yapılan hesaplamayı temel model bir Turing makinesi ile yapmak ya da tanınan dili temel model bir Turing makinesi ile tanımak mümkündür. Turing makinesi modelinde yapılabilecek değişiklerden başlıcaları aşağıdaki gibi sıralanabilir.

1. İki yönlü sonsuz şerit kullanan Turing makinesi

Temel modelde sol ucu sınırlı, sağ ucu sınırsız olan şeridin bu modelde her iki ucu da sonsuzdur. Başlangıçta şerit üzerinde belirli bir giriş dizgisi kayıtlıdır. Şerit üzerinde giriş dizgisinin sol ve sağ taraflarının sonsuz sayıda **B** ile dolu olduğu varsayılır.

2. Çok şeritli Turing makinesi

Bu modelde makinenin bir sonlu denetim birimi, ancak n adet şeridi ve n adet okuma kafası vardır. Başlangıçta giriş dizgisi şeritlerden birinde kayıtlıdır. Diğer şeritler ise boştur. Her harekette, Turing makinesi her şeritten bir simge okur, her şeride bir simge yazar ve okuma kafalarının her biri bir sağdaki ya da bir soldaki hücreye geçer. Şeritler üzerindeki yazma ve sağa/sola geçme hareketleri birbirinden bağımsızdır. Ancak sonlu denetim birimi tek olduğu için durum geçişleri ortaktır.

3. Çok izli Turing makinesi

Bu modelde makinenin bir sonlu denetim birimi, n izli bir şeridi, n izli şerit üzerinde okuma ve yazma yapabilen bir de okuma kafası vardır. Her harekette makine her izden bir tane olmak üzere n simge okur, her ize bir tane olmak üzere n simge yazar; okuma kafası bir sağa ya da bir sola, sonlu denetim birimi de yeni bir duruma geçer.

4. Birden çok okuma kafası bulunan Turing makinesi

Bu modelde makinenin bir sonlu denetim birimi, tek izli bir şeridi, birden çok da okuma kafası bulunur. Belirli bir anda okuma kafalarının her biri şeridin belli bir hücresi üzerindedir. Her harekette okuma kafalarının her biri bulunduğu hücredeki simgeyi okur, yerine bir simge yazar ve bir sağdaki/soldaki hücreye geçer.

5. Çok boyutlu Turing makinesi

Temel modelde tek boyutlu olan okuma şeridi bu modelde çok boyutludur. Örneğin k boyutlu olan şerit 2k yönde sonsuzdur. Her harekette okuma kafası

üzerinde bulunduğu hücredeki simgeyi okuyup, yerine yeni bir simge yazdıktan sonra 2k yönden birinde ilerler.

6. *Off-line* Turing makinesi

Çok şeritli modelde olduğu gibi bu modelde de makinenin birden çok şeridi vardır. Ancak şeritlerden biri giriş şerididir ve bu şerit üzerinde yalnız okuma yapılabilir. Başlangıçta giriş şeridi üzerinde bir dizgi kayıtlıdır, diğer şeritler ise boştur. Makine her hareketinde her şeritten bir simge okur, giriş şeridi dışındaki şeritlere birer simge yazar ve her şerit üzerindeki okuma kafası bir sağ ya da sol hücreye geçer.

7. Deterministik olmayan Turing makinesi

Turing makinesinin temel modeli deterministik bir modeldir. Deterministik modelde her anlık tanıma en çok bir hareket eşlenir. Deterministik olmayan modelde ise her anlık tanıma sıfır, bir ya da birden çok hareket eşlenebilir.

6.4. Chomsky Sıradüzeni

Sonlu özdevinir, yığıtlı özdevinir (PDA) ve Turing makinesi adlarıyla sunulan üç modelden birincisi sonlu bellekli, diğer ikisi ise sonsuz bellekli modellerdir. Sonlu özdevinir modelinde bilgi saklamak için kullanılan açık bir birim yoktur. Modelin saklama kapasitesi durumlarla sınırlıdır. Durum sayısı sınırlı olduğu için de model sonlu bellekli bir modeldir. Yığıtlı özdevinir ve Turing makinesi modellerinde ise bilgi saklamak için kullanılan sonsuz kapasiteli birer birim (yığıt ve şerit) vardır. Sonsuz bellekli modellerin işlem gücünün sonlu bellekli modelden daha büyük olduğu açıktır. Sonsuz bellekli modellerde ise, Turing makinesinin gücü yığıtlı özdevinirden daha büyüktür. Yığıtlı özdevinir ve Turing makinesi modellerinin işlem gücü açısından farkı bellek kapasitesinden değil, modelin yapısından kaynaklanmaktadır. Turing makinesi modeli, yığıtlı özdevinir modeline göre daha güçlü bir modeldir.

Biçimsel diller ve biçimsel dillere karşı gelen makine modellerini aşağıdaki gibi sıralayabiliriz :

Biçimsel Dil Sınıfı	Makine Modeli
Tür-3 ya da düzgün dil	Sonlu Özdevinir (FA)
Tür-2 ya da bağlamdan-bağımsız dil	Yığıtlı Özdevinir (PDA)
Tür-1 ya da bağlama-bağımlı dil	Doğrusal-Sınırlı Özdevinir (LBA)
Tür-0 ya da kısıtlamasız dil	Turing makinesi (TM)

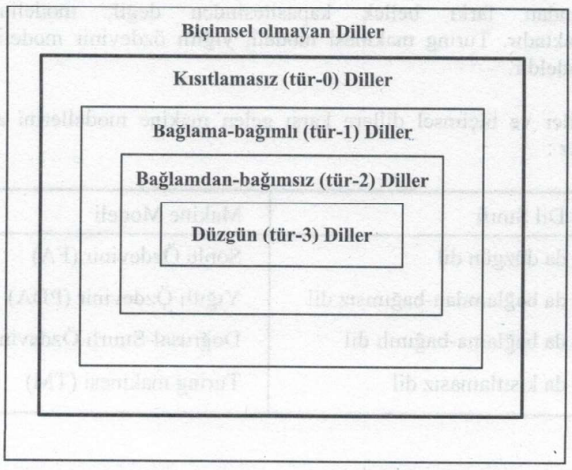
Yukarıdaki çizelgede yer alan biçimsel dil sınıflarının tümü ile makine sınıflarının üçü bu kitapta incelenmektedir. Doğrusal-sınırlı özdevinirler (*linear-bounded automata* – LBA) olarak adlandırılan makine modeli ise, diğer üç model kadar yaygın kullanılan bir model olmadığı için bu kitap kapsamına alınmamıştır. Çok kısa olarak LBA modeli, Turing makinesi modelinin şerit kapasitesi sınırlı bir biçimidir ve şerit kapasitesindeki sınırlama giriş dizgisi tarafından belirlenir.

Biçimsel dil ve makine sınıfları arasındaki ilişki ise aşağıdaki gibi özetlenebilir :

1. Sonlu özdevinirler düzgün dilleri tanıyan makinelerdir.
2. Yığıtlı özdevinirler bağlamdan-bağımsız dilleri tanıyan makinelerdir.
3. Doğrusal-sınırlı özdevinirler bağlama-bağımlı dilleri tanıyan makinelerdir.
4. Turing makineleri ise kısıtlamasız dilleri tanıyan makinelerdir.

Yukarıda sıralanan dil sınıfları birbirinden bağımsız değildir. Biçimsel dil sınıfları arasındaki sıradüzen Chomsky sıradüzeni (*Chomsky hierarchy*) olarak bilinir ve kısaca aşağıdaki gibi ifade edilebilir (Çizim 6.2):

1. Her bağlama-bağımlı dil aynı zamanda bir kısıtlamasız dildir. Başka bir deyişle bağlama-bağımlı dil sınıfı kısıtlamasız dil sınıfının bir altsınıfıdır.
2. Her bağlamdan-bağımsız dil aynı zamanda bir bağlama-bağımlı dildir. Başka bir deyişle bağlamdan-bağımsız dil sınıfı, bağlama-bağımlı dil sınıfının bir altsınıfıdır.
3. Her Düzgün dil aynı zamanda bir bağlamdan-bağımsız dildir. Başka bir deyişle düzgün diller sınıfı, bağlamdan-bağımsız dil sınıfının bir altsınıfıdır.



Çizim 6.2. Biçimsel Dil Sınıfları Sıradüzeni : Chomsky Sıradüzeni